MEMORY - Product Blueprint & Implementation Plan

iQOO Hackathon 2026

1. The Product

Name: MEMORY **Tagline:** *Your phone remembers. You just have to ask.*

Official definition: MEMORY is an on-device Personal Memory Layer that turns a user's photos, screenshots and documents into a private, searchable memory.

One-line pitch: MEMORY is a private, on-device AI memory layer that lets you search your selected photos, screenshots and documents using human language - not filenames, folders, or endless scrolling.

The core insight: Conventional storage understands *filename + folder + timestamp + file type*. Humans remember *person + place + event + meaning*. MEMORY bridges that gap by building a private semantic representation of a user's selected personal storage - photos, screenshots, documents - and letting them retrieve it by describing what they remember, e.g. *"Show me photos of Mom at the temple"* or *"Find the restaurant John sent me."*

Positioning: Not an AI gallery, not a file manager, not "just semantic search." It's a **Personal Memory Layer for the phone** - *"We don't organize your files. We understand why you remember them."*

Track: Submit under **Productivity** ("build AI-powered solutions that help people work smarter, automate repetitive tasks, manage time and information, improve workflows, and get more done") - MEMORY is a direct fit. Open Innovation is the fallback if needed.

Why this fits the competition (official Chennai City Battle rubric)

Scoring is one rubric across all four cities: **75% Jury Panel + 25% HackTracker device data** (device data is logged automatically from the phone/laptop, not self-reported).

Criterion	Weight	Source	How MEMORY delivers
End product quality	30%	Jury	Real working retrieval on real files - does it work, is it useful, would someone keep using it
Novelty and impact	20%	Jury	Cross-media personal memory is the differentiated angle, not

			“another AI gallery”
Creative phone use	15%	HackTracker (device)	Real on-device compute verified in use; camera/voice claimed only if genuinely implemented
Technical depth	15%	Jury	Hybrid retrieval architecture, real engineering trade-offs, real use of the hardware
Office Kit usage	10%	HackTracker (device)	Actually pair and use Office Kit during Red Light - usage counts/durations are logged
Demo and presentation	10%	Jury	Tight 3-5 minute pitch, live and real

Because 25% of the score is **logged device data, not self-reporting**, the working rule is: don’t just claim on-device AI in the pitch - actually route meaningful AI workloads through the phone’s on-device compute stack and verify actual device-side execution, and actually pair/use Office Kit, since that’s exactly what HackTracker measures. Camera and voice are only claimed if the product genuinely implements them (e.g. a real voice query → speech-to-text → query parser → retrieval path) - not invented just because the rubric mentions them; if the hackathon build only takes typed queries over stored photos/screenshots/documents, that’s what gets claimed.

Non-Negotiable Product Principles

1. **Real data only** - no hardcoded demo results, no query→answer mappings.
2. **Local-first** - core intelligence works without cloud services.
3. **Evidence-grounded** - the system never invents why a result matched.
4. **Incremental** - new/changed files are indexed without rebuilding everything.
5. **Explainable** - every result can expose its matching signals.
6. **Privacy-preserving** - personal data and embeddings remain on-device.
7. **Graceful failure** - failures produce real error states, never fake results.

Feature hierarchy

Presenting five features as a flat list is harder for judges to parse than a clear hierarchy:

- **Core:** Natural-language memory search.
- **Differentiators:** Personal identity, cross-media retrieval.
- **Trust:** On-device, offline, explainable results.
- **Supporting intelligence:** OCR, metadata, embeddings, reranking.

The five MVP capabilities (hackathon scope)

1. **Semantic photo search** - natural-language queries over visual content and metadata.
2. **Document search** - OCR + metadata + embeddings (e.g. “my latest software engineering resume”).
3. **Cross-media search (THE hero feature - lead the pitch and demo with this)** - a screenshot’s extracted text resolves to a real file through an explicit evidence chain, not an unexplained jump: *“Find the restaurant John sent me”* → `John (person) → screenshot (file) → OCR: restaurant name → restaurant entity → shared location/date context → related indexed files`. Each arrow is a real, inspectable retrieval step - the product performs explainable retrieval across evidence, it doesn’t “understand a trip.” This is more differentiated than “find photos of Mom” alone and should anchor the whole narrative.
4. **Personal identity** - user registers “Mom” from a few real photos; later queries like *“Mom at the beach”* resolve via local face embeddings.
5. **Offline demonstration** - airplane mode, search still works. Proves the privacy claim isn’t marketing.

Deliberately cut for the hackathon (become roadmap items): full video understanding, continuous whole-device indexing, automatic cleanup, best-shot engine, relationship graphs, large persistent LLM, autonomous agent behavior. A fully general “show me everything from my Chennai trip” query - which implies event/timeline understanding - is a strong **future** capability, not a hackathon-guaranteed demo; the V1-realistic version is “show me the restaurant screenshot and documents related to my Chennai trip,” built from real evidence links (shared location/date/entity), not a claimed event graph.

Competitive moat

No single technology here is novel (OCR, embeddings, face-matching all exist independently). The moat is the **combination**: personal identity + cross-media memory + semantic retrieval + on-device privacy + phone-native execution, wrapped in one coherent product experience.

---

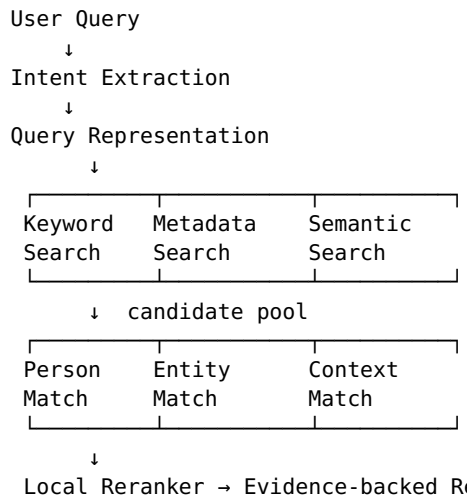
2. System Architecture

The Memory Object (core abstraction)

The architecture is organized around **files** below, but the product concept it serves is a **Memory**: the semantic representation of one or more pieces of user evidence, which may span files, people, locations, timestamps, OCR text, visual concepts and relationships. This is what makes cross-media retrieval technically coherent rather than a coincidence of separate searches:

```
Memory: "Restaurant John sent"
├── Screenshot (file)
├── OCR text
├── Person: John
├── Location: Chennai
└── Restaurant entity
```

Retrieval pipeline (query time)



Design principle: don't route every query through an LLM. Use the cheapest reliable signal first (keyword/metadata), and reserve heavier AI (semantic embeddings, reranking) for ambiguity. This is the trade-off that demonstrates real engineering judgment on mobile hardware.

No-hallucination rule (elevated to a product requirement): MEMORY never generates factual answers independently of indexed evidence. The model's job is only to structure query intent - never to invent facts; the user's indexed data supplies the actual evidence.

Retrieval confidence

Every result carries an explicit confidence level, not a false sense of certainty:

- **High** - `Matched because: Mom · Temple · Chennai`
- **Medium/Low** - surfaced as "Possibly related memories," visually distinct from high-confidence hits
- **No match** - if a query has no sufficiently supported evidence (e.g. "photos of my dog at the beach" with no dog photos indexed), MEMORY says so rather than returning vaguely similar results to avoid an empty screen. This negative-query behavior is tested explicitly (see Testing & quality gates).

Indexing pipeline (ingest time)

```
File → type detection → cheap metadata extraction → duplicate check
     → AI processing only if necessary → embeddings/OCR/visual
features
     → local index (SQLite = source of truth; vector index =
optimization)
```

Priority tiers: **P0** files needed for the current query → **P1** recently modified → **P2** recent photos/docs → **P3** older content → **P4** low-value media. This gives *progressive usefulness* - the app is searchable before indexing finishes.

Local data model (Room/SQLite)

Core tables: `files` (uri, mime, size, timestamps, geo, hash, status), `people` + `person_embeddings` + `file_people` (identity matches with confidence), `document_chunks` + `embeddings` (page/offset-aware text

chunks), `visual_features` (OCR text, objects, scene, quality score). Every embedding record also carries `model_version`, `embedding_dimension`, and `created_at` - if the embedding model is later swapped, affected vectors can be identified and regenerated rather than silently going stale.

AI model interfaces (swappable, not hard-coded)

```
        interface ImageEmbeddingModel { suspend fun embed(image: Bitmap):
FloatArray }
        interface TextEmbeddingModel  { suspend fun embed(text: String):
FloatArray }
        interface OcrEngine           { suspend fun extract(doc:
InputStream): OcrResult }
        interface FaceEmbeddingModel  { suspend fun embed(face: Bitmap):
FloatArray }
        interface VectorIndex {
            suspend fun add(...); suspend fun remove(...)
            suspend fun search(...); suspend fun rebuild(...)
        }
```

Treating models as implementation details means swapping in a better model or accelerator later doesn't require redesigning the product.

Ranking (tunable, not fixed)

```
score = semantic_similarity(0.40) + metadata_match(0.20) +
person_match(0.20)
      + text_match(0.10) + contextual_match(0.10)
```

Weights start as a hypothesis and get tuned against a benchmark query set (Precision@5, Recall@10, MRR, latency) - not shipped as guesses.

"Why this matched" - always evidence-based

Results are explained from actual indexed evidence (`Matched because: Mom · Temple · Chennai`, or `Matched text: "software engineering intern"`), never a generic AI-generated explanation. This is what makes the system trustworthy rather than a black box.

---

3. Privacy Architecture

Privacy is the product's identity, not a checkbox feature.

- All embeddings, OCR text, identities, and search history are stored **on-device only**; no cloud upload path for core retrieval.
- Sensitive local storage encrypted; keys managed via **Android Keystore**.
- Logs never contain OCR text, personal names, document contents, embeddings, or unnecessary file paths.
- UI carries a persistent *** Local AI** indicator, and a settings screen makes the on-device guarantee explicit and auditable - only claiming what the implementation actually does.
- **Deletion lifecycle:** when a user removes a folder/file from

MEMORY, all associated metadata, OCR text, embeddings, thumbnails, identity associations, and vector-index entries are removed or invalidated - not just the file record.
- All core retrieval, embeddings, OCR, identity matching, and search operate locally on the device. Cloud services are not required for the core product.

---

4. Real, Non-Simulated Implementation Plan

The hackathon build delivers a **genuinely functional vertical slice** using real user-selected files, real on-device AI, and real retrieval - no hardcoded query→result mappings, no fake loading states, no cloud secretly doing the work. The architecture is designed for production extension beyond the hackathon; the 25-hour build is not claimed to be production-ready on its own, but every slice that ships is real, not staged.

Tech stack

Kotlin + Jetpack Compose, Coroutines/Flow, WorkManager (background indexing), Room (source of truth), Android Storage Access Framework + MediaStore (real file access), Android Keystore (encryption).

Ingestion (real files, no fake dataset)

```
User selects folder (SAF) → persist URI permission → enumerate files
   → filter supported MIME types → create file records → queue
indexing
```

Incremental indexing

Every file is hashed and compared against the DB; unchanged files are skipped entirely. States: `DISCOVERED → QUEUED → PROCESSING → INDEXED / FAILED / STALE` - no silent failures.

Staged processing cost

1. **Cheap:** filename, MIME, size, timestamp, EXIF, hash.
2. **Moderate:** thumbnail, dimensions, perceptual hash.
3. **AI (only if needed):** embeddings, OCR, face detection, scene/object recognition.

Query understanding (small local model, constrained)

The model's job is **only** to structure intent - never to invent facts:

```
"Show me photos of Mom at the temple in Chennai"
→ { "media_type": "image", "person": "Mom", "scene": "temple",
"location": "Chennai" }
```

The retrieval engine - not the model - supplies the actual evidence.

Person registration (explicit, confidence-gated)

User registers a person from several real photos → face detection → multiple stored embeddings per person (not one averaged vector). At index time, faces below a confidence threshold are marked *unknown* rather than guessed.

Performance targets

- Simple metadata/keyword queries: **< 500 ms**
- Typical semantic queries: **~1 second**
- Indexing: asynchronous and interruptible; searching must never feel asynchronous.
- No permanent multi-model memory footprint - models are loaded on demand and released/cached strategically (target class: ~8 GB RAM devices, quantized models).
- Thermal/battery awareness: indexing concurrency backs off under thermal load; search stays high priority, background indexing does not.

Testing & quality gates

An automated corpus (`/testdata` with photos, documents, screenshots, and a `queries.json` of expected results) drives evaluation across 100+ natural-language queries, measuring Precision@5, Recall@10, MRR, latency, and crash rate. The corpus explicitly includes **negative queries** - e.g. “photos of my dog at the beach” with no dog photos indexed - where the required behavior is an honest “no sufficiently supported match,” not a vaguely similar result returned just to avoid an empty screen.

Gate	Bar
Ingestion	99%+ of supported files discovered
Indexing	No silent failures
Search	Relevant result in top 5 for most benchmark queries
Negative search	No confident-looking result returned when no real match exists
Latency	~1s or less for indexed queries
Offline	Core retrieval works with network disabled
Restart	Index persists across app restarts
Incremental	New file becomes searchable without full reindex
Privacy	No core personal data required to leave device

Definition of Done (hackathon build)

- ☐ Real user files can be imported.
- ☐ Files persist across app restarts.
- ☐ No hardcoded query/result mappings exist.
- ☐ Core search works offline.

- [ ] Search returns actual indexed files.
- [ ] OCR results come from actual documents/screenshots.
- [ ] Embeddings are generated on-device.
- [ ] New files can be incrementally indexed.
- [ ] Results show evidence for the match, with a confidence level.
- [ ] Negative queries return an honest “no match,” not a forced result.
- [ ] Failed processing produces a real error state.
- [ ] No cloud service is required for core search.
- [ ] App can open the actual source file.
- [ ] Removing a file/folder invalidates its metadata, embeddings, thumbnails, and index entries.

Build sequence (vertical slices, each one fully working)

Must ship: real file ingestion, metadata extraction, image embeddings, OCR, local text/semantic search, hybrid retrieval, result explanation with confidence, offline search, actual file opening.

Should ship: person registration, person retrieval, cross-media retrieval, incremental indexing, query parsing.

Stretch: advanced reranking, event grouping, duplicate detection, sophisticated entity extraction, video.

Sequenced as vertical slices:

1. Storage → Room database (no AI yet)
2. Database → keyword/metadata search
3. Image → embeddings → semantic search (“temple” works)
4. Documents → OCR → semantic search (“resume” works)
5. Person registration (“Mom” works)
6. Query parser combining signals (“Mom + temple + Chennai”)
7. Cross-media fusion (screenshot OCR ↔ photos ↔ documents)
8. Optimization pass: quantization, caching, batching, hardware acceleration, thermal handling

Roadmap (believable evolution from hackathon to platform)

- **Hackathon V1:** Memory Search (must/should items above)
- **V1.1:** Personal Identity + Cross-Media Memory hardened (duplicate detection, similar-photo grouping, blur detection)
- **V1.2:** Events + Timeline (video frame indexing, audio transcription, full general trip/event queries)
- **V2:** Memory Graph (relationships, richer context)
- **V3:** Proactive Personal Memory
- **Later:** Personal AI Agent - explicitly a distant vision, not a claim made about the hackathon build (“MEMORY = personal memory retrieval system,” not “MEMORY = autonomous phone agent”)

---

5. Technical Design Specification

This section converts the blueprint above into a concrete, implementation-ready spec for the iQOO Android build. Where the source material doesn’t give enough basis to commit to an exact choice, the item is marked **[VERIFY]** - to be confirmed by benchmarking on the actual loaner device rather than assumed.

5.1 Exact AI models

Capability	Model	Format	Size (approx)	Dim
Image embeddings	MediaPipe Image Embedder, MobileNetV3-Small backbone	TFLite, INT8 post-training quantized	~10-15 MB	1024 (or 256 if EfficientL Lite0 variant used)
Text embeddings	all-MiniLM-L6-v2 exported to TFLite [VERIFY export pipeline on-device before committing]	TFLite, INT8	~23 MB quantized	384
OCR	ML Kit Text Recognition v2 (on-device)	Play Services bundled model	Downloaded on demand (~15-30 MB)	n/a
Face detection	ML Kit Face Detection (on-device)	Play Services bundled model	Downloaded on demand	n/a
Face embedding	MobileFaceNet, TFLite	TFLite, INT8 [VERIFY license terms of the specific pretrained weights used]	~5 MB	128
Query understanding	Rule-based/regex + keyword extraction (primary); Gemma 3 1B via MediaPipe LLM Inference API (stretch only) [VERIFY RAM headroom]	TFLite (if LLM used)	~500 MB-1 GB quantized if LLM used	n/a

Constraint respected: no model is claimed as “using the NPU” until NNAPI delegate acceleration is confirmed at runtime on the actual device (see 5.3).

5.2 Exact Android APIs actually used

Need	API
Folder selection	`Intent.ACTION_OPEN_DOCUMENT_TREE`
Persistent access	`ContentResolver.takePersistableUriPermission()`
File discovery	`DocumentFile.listFiles()` walked recursively from the granted tree URI
Change detection	`ContentObserver` registered on `MediaStore.Files.getContentUri("external")` for new/changed media
Image decoding	`ImageDecoder` (API 28+) with `ImageDecoder.Source`
PDF access	`PdfRenderer` for rendering pages when no text layer exists (feeds OCR); direct byte extraction for text-layer PDFs
Background work	`WorkManager` with `CoroutineWorker`, constraints: `setRequiresBatteryNotLow(true)`, `setRequiresStorageNotLow(true)`
Thermal awareness	`PowerManager.getCurrentThermalStatus()` + `PowerManager.addThermalStatusListener()` (API 29+)
Network detection	`ConnectivityManager.NetworkCallback` (used only to confirm offline mode during the demo, not required for core function)
Encryption keys	`Android Keystore` via `KeyGenParameterSpec` (AES/GCM)
Encrypted local storage	Jetpack Security Crypto `EncryptedFile` / `EncryptedSharedPreferences`, backed by the Keystore-managed key
App-private storage	`context.filesDir` / `context.getExternalFilesDir(null)` for thumbnails and derived caches
Progress reporting	`WorkManager.setProgressAsync()` surfaced through a `Flow` to the UI; `NotificationCompat` for a foreground indexing notification
Opening source files	`FileProvider` + `Intent.ACTION_VIEW` with the original document/media URI

5.3 Exact on-device inference runtime

- **Runtime:** TensorFlow Lite (LiteRT) via `com.google.android.gms:play-services-tflite` (Google Play services delivery, avoids bundling large native libs in the APK).
- **Acceleration strategy:** try **NNAPI delegate** first (`NnApiDelegate`) → fall back to **GPU delegate** → fall back to **CPU with XNNPACK**. The chosen delegate is logged per model at first run.
- **Verification requirement [VERIFY on loaner device]:** acceleration is only claimed in the pitch if `Interpreter.Options().setUseNNAPI(true)` succeeds without falling back, confirmed by comparing inference latency across delegate paths on the actual iQOO 15 before the event or during the Saturday teach-in window.
- **Threading:** each model type (image embed, text embed, OCR, face detect, face embed) gets one `Interpreter` instance behind a single-threaded dispatcher (`Dispatchers.Default.limitedParallelism(1)` per model) to avoid concurrent-access crashes; separate models can run in parallel across different dispatchers.
- **Model lifecycle:** lazy-loaded on first use, kept warm during an

active indexing/search session, released via `interpreter.close()` on `ComponentCallbacks2.onTrimMemory(TRIM_MEMORY_MODERATE)` or higher.
- **Quantization:** INT8 post-training quantization for all bundled TFLite models to minimize both load time and memory footprint.
- **Concurrent inference limit:** at most 2 models running inference simultaneously (e.g. OCR + text embedding), enforced by a shared semaphore, to bound peak memory and thermal load.

5.4 Exact vector index

At hackathon dataset scale (150-2,000 files, low thousands of vectors), an external ANN library is unnecessary complexity. Chosen approach:

- **Implementation:** flat in-memory index - a `FloatArray`-backed structure loaded from Room at app start (and incrementally updated on writes), with brute-force cosine similarity computed against the query vector.
- **Storage format:** vectors persisted as `BLOB` (serialized `float32[]`) in the Room `embeddings` table; Room/SQLite remains the canonical source of truth, the in-memory index is a rebuildable cache.
- **Similarity metric:** cosine similarity.
- **Search:** exact (not approximate) top-K via a partial sort; **[VERIFY]** expected latency <50ms for up to ~5,000 vectors on the target CPU - to be confirmed once the demo dataset is finalized.
- **Update behavior:** insert/update/delete apply directly to the in-memory array plus the Room row in the same transaction; a full rebuild is only needed after a model-version change (see 5.5, `model_version`).
- **Upgrade path [VERIFY if needed]:** if the dataset ever exceeds roughly 50,000 vectors, evaluate ObjectBox Vector or a similar on-device ANN library - explicitly out of scope for the hackathon.

5.5 Exact Room / SQLite schema

```
        -- Canonical file record
        CREATE TABLE files (
          id TEXT PRIMARY KEY,
          uri TEXT NOT NULL,
          mime_type TEXT NOT NULL,
          filename TEXT,
          size_bytes INTEGER,
          created_at INTEGER,
          modified_at INTEGER,
          latitude REAL,
          longitude REAL,
          width INTEGER,
          height INTEGER,
          duration_ms INTEGER,
          sha256 TEXT NOT NULL,
          status TEXT NOT NULL,      --
DISCOVERED|QUEUED|PROCESSING|INDEXED|FAILED|STALE
          indexed_at INTEGER,
          UNIQUE(sha256)
        );
        CREATE INDEX idx_files_status ON files(status);
        CREATE INDEX idx_files_modified ON files(modified_at);

        CREATE TABLE processing_errors (
          id TEXT PRIMARY KEY,
          file_id TEXT NOT NULL REFERENCES files(id) ON DELETE CASCADE,
          stage TEXT NOT NULL,       -- e.g. OCR, EMBEDDING, FACE_DETECT
          message TEXT,
          occurred_at INTEGER,
```

```
  retry_count INTEGER DEFAULT 0
);

CREATE TABLE people (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  created_at INTEGER
);

CREATE TABLE person_embeddings (
  id TEXT PRIMARY KEY,
  person_id TEXT NOT NULL REFERENCES people(id) ON DELETE CASCADE,
  vector BLOB NOT NULL,
  model_version TEXT NOT NULL,
  source_file_id TEXT REFERENCES files(id)
);

CREATE TABLE file_people (
  file_id TEXT NOT NULL REFERENCES files(id) ON DELETE CASCADE,
  person_id TEXT NOT NULL REFERENCES people(id) ON DELETE CASCADE,
  confidence REAL NOT NULL,
  PRIMARY KEY (file_id, person_id)
);

CREATE TABLE document_chunks (
  id TEXT PRIMARY KEY,
  file_id TEXT NOT NULL REFERENCES files(id) ON DELETE CASCADE,
  page INTEGER,
  text TEXT NOT NULL,
  start_offset INTEGER,
  end_offset INTEGER
);

CREATE TABLE embeddings (
  id TEXT PRIMARY KEY,
  file_id TEXT REFERENCES files(id) ON DELETE CASCADE,
  chunk_id TEXT REFERENCES document_chunks(id) ON DELETE CASCADE,
  vector BLOB NOT NULL,
  model_version TEXT NOT NULL,
  embedding_dimension INTEGER NOT NULL,
  created_at INTEGER NOT NULL
);
CREATE INDEX idx_embeddings_file ON embeddings(file_id);
CREATE INDEX idx_embeddings_model ON embeddings(model_version);

CREATE TABLE visual_features (
  file_id TEXT PRIMARY KEY REFERENCES files(id) ON DELETE CASCADE,
  ocr_text TEXT,
  objects TEXT,              -- JSON array
  scene TEXT,
  quality_score REAL
);

CREATE TABLE locations (
  id TEXT PRIMARY KEY,
  file_id TEXT NOT NULL REFERENCES files(id) ON DELETE CASCADE,
  coarse_name TEXT,          -- e.g. "Chennai" - resolved
locally/offline where possible
  latitude REAL,
  longitude REAL
);

-- Memory Object layer (ties multi-file evidence together)
CREATE TABLE memories (
  id TEXT PRIMARY KEY,
  label TEXT,                -- e.g. "Restaurant John sent"
```

```
  created_at INTEGER
);

CREATE TABLE memory_evidence (
  memory_id TEXT NOT NULL REFERENCES memories(id) ON DELETE CASCADE,
  file_id TEXT NOT NULL REFERENCES files(id) ON DELETE CASCADE,
  evidence_type TEXT NOT NULL,  -- PERSON|OCR_ENTITY|LOCATION|DATE|SEMANTIC
  evidence_value TEXT,
  PRIMARY KEY (memory_id, file_id, evidence_type)
);
```

Migration strategy: Room `AutoMigration` for additive changes (new columns/tables); destructive migrations avoided during the hackathon by treating `model_version` as the trigger for re-embedding rather than schema changes.

5.6 Exact application module structure

```
app/
├── presentation/     -- Compose UI, ViewModels, search/results screens
├── domain/           -- QueryOrchestrator, MemoryService, IndexingService (use-case layer)
├── data/              -- Room DAOs/entities, repositories
├── ingestion/          -- SAF folder selection, MediaStore observer, file enumeration
├── indexing/           -- WorkManager workers, staged processing (cheap -> moderate -> AI)
├── retrieval/          -- keyword/metadata/semantic search, fusion, reranking, confidence
├── ai/                 -- model interfaces + TFLite/ML Kit implementations, delegate selection
├── storage/            -- vector index (in-memory + persistence), thumbnail cache
├── security/           -- Keystore key management, EncryptedFile wrappers
└── benchmark/          -- test corpus runner, Precision@5/Recall@10/MRR harness
```

Each module exposes only interfaces to `domain/` (e.g. `ai/` exposes `ImageEmbeddingModel`, not TFLite types); `domain/` never imports concrete implementations directly, so swapping a model or the vector index doesn't ripple into the UI or business logic.

5.7 Exact query pipeline

```
User Query
    ↓
Query Parsing (regex/keyword extraction; LLM only if stretch path enabled)
    ↓
Intent Representation  {media_type, person, scene, location, keywords}
    ↓
Keyword Retrieval (SQLite LIKE / FTS on filename + OCR text)
    ↓
Metadata Retrieval (date range, mime type, geo match)
    ↓
Semantic Retrieval (cosine similarity over in-memory vector index, top-K=50)
    ↓
Person/Entity Filtering (file_people confidence >= threshold)
    ↓
```

```
Candidate Fusion (union of the above, deduplicated by file_id)
    ↓
Reranking (weighted score, see 5.8)
    ↓
Confidence Evaluation (High >=0.75 / Medium 0.5-0.75 / Low <0.5 /
No-match <0.3) [VERIFY thresholds by benchmarking]
    ↓
Evidence Generation ("Matched because: ..." built from the actual
signals that fired)
    ↓
UI Results
```

AI is confined to the **Query Parsing** and **Semantic Retrieval** steps; every other step operates on indexed, real evidence, so the model can never fabricate why a result matched.

5.8 Exact ranking formula

```
score = 0.40 * semantic_similarity
      + 0.20 * metadata_match
      + 0.20 * person_match
      + 0.10 * text_match
      + 0.10 * contextual_match (location/date overlap)
```

- All signals normalized to [0,1] before weighting.
- These weights are a **starting hypothesis**, not a final answer - tuned against the benchmark corpus (5.14) once real Precision@5/Recall@10 numbers exist.

5.9 Exact model / memory budget (targets, [VERIFY] on device)

Resource	Target
Total resident model memory (all loaded models)	< 200 MB at any one time
Vector index (in-memory)	< 10 MB at hackathon dataset scale (~2,000 vectors x 384 dim x 4 bytes ≈ 3 MB)
Room database size	< 50 MB for the demo dataset
Thumbnail/cache size	< 100 MB, LRU-evicted
Max concurrent inference jobs	2
Image resolution fed to embedding model	resized to model's native input (e.g. 224x224) before inference
Indexing throughput	[VERIFY] target ~5-10 files/sec for cheap-stage metadata, ~1-2 files/sec for AI-stage processing

Under memory pressure: `onTrimMemory` triggers releasing non-active model interpreters first, then shrinking the thumbnail cache; the vector index and Room DB are never evicted since they're required for search to keep working.

Under thermal pressure: background indexing concurrency is reduced (fewer parallel WorkManager jobs); active search inference is never throttled.

5.10 Exact threading and concurrency model

- **UI/main thread:** rendering only; never touches inference, disk I/O, or Room queries directly.
- **IO dispatcher (`Dispatchers.IO`):** file reads, Room queries, SAF/MediaStore calls.
- **Inference dispatchers:** one `limitedParallelism(1)` dispatcher per model type (5.3).
- **Background indexing workers:** WorkManager `CoroutineWorker` instances, each processing one file at a time, throttled by the concurrent-inference semaphore.
- **Vector search:** runs on `Dispatchers.Default` (CPU-bound, no I/O).
- **Cancellation:** every worker checks `isStopped`/coroutine cancellation between pipeline stages so a cancelled indexing job doesn't leave a file half-processed.
- **Prioritization:** an active user search always preempts background indexing for CPU/inference dispatcher access (search requests jump the inference queue).

5.11 Exact indexing pipeline

```
File Discovery (SAF enumeration / MediaStore ContentObserver)
    ↓
Metadata Extraction (filename, mime, size, timestamp, EXIF)
    ↓
Hash / Duplicate Check (SHA-256 against files.sha256 UNIQUE
constraint)
    ↓ unchanged -> SKIP
Media Classification (image / document / screenshot heuristic)
    ↓
OCR / Vision / Embedding (staged: cheap -> moderate -> AI, per 5.9)
    ↓
Structured Feature Extraction (visual_features, document_chunks,
locations)
    ↓
Room Persistence (transactional write)
    ↓
Vector Index Update (in-memory + embeddings table)
    ↓
INDEXED
```

- **States:** `DISCOVERED → QUEUED → PROCESSING → INDEXED | FAILED | STALE`.
- **Retry:** up to 2 automatic retries on transient failure (logged in `processing_errors`), then marked `FAILED` with a visible, real error state - never silently dropped.
- **Incremental updates:** hash comparison ensures unchanged files are always skipped; changed files are reprocessed from Metadata Extraction onward.
- **Deleted files:** MediaStore/SAF permission revocation triggers deletion of the file row and cascades (via `ON DELETE CASCADE`) through embeddings, OCR text, face associations, and memory evidence.
- **Model-version changes:** a version bump in `embeddings.model_version` marks affected rows `STALE`, queued for re-embedding without touching unaffected rows.
- **Restart recovery:** any file left `PROCESSING` at app start is re-queued.

5.12 Exact privacy and security implementation

- **What's stored:** file metadata, OCR text, embeddings, face embeddings, thumbnails, search history - all in the app-private

Room database and app-private file storage.

- **What's encrypted:** the Room database file and thumbnail cache, via Jetpack Security Crypto `EncryptedFile`, keyed by a Keystore-backed `MasterKey` (AES256-GCM).
- **Key lifecycle:** key generated on first launch via `KeyGenParameterSpec`, non-exportable, hardware-backed where the device supports it.
- **Logging:** no OCR text, names, document contents, embeddings, or full file paths in logs - only file IDs and stage names.
- **Deletion:** removing a folder/file cascades through embeddings, OCR text, thumbnails, identity associations, and vector-index entries (5.5, 5.11).
- **No-cloud confirmation:** core search, embedding, OCR, and identity matching have zero required network calls, verifiable by running the full query pipeline in airplane mode.

5.13 Exact performance targets (target vs measured)

Metric	Target	Measured
App startup (cold)	< 2s	[VERIFY]
Simple metadata/keyword query	< 500 ms	[VERIFY]
Semantic query (incl. rerank)	< 1 s	[VERIFY]
Indexing throughput (cheap stage)	5-10 files/sec	[VERIFY]
Indexing throughput (AI stage)	1-2 files/sec	[VERIFY]
OCR throughput	[VERIFY]	[VERIFY]
Peak memory (all models resident)	< 200 MB	[VERIFY]
Battery drain during 30-min indexing	[VERIFY]	[VERIFY]
Thermal state during sustained indexing	stays below device's "severe" threshold	[VERIFY]

The "Measured" column is filled in during the Hours 18-22 performance pass (see Section 6) and reported honestly in the pitch - target numbers are never presented as achieved results until actually measured on the loaner device.

5.14 Exact benchmark and acceptance tests

Corpus: `/testdata` (150-300 curated files matching the demo dataset) + `queries.json` (100+ natural-language queries with expected file IDs), covering:

- semantic photo search, person search, OCR/document search, cross-media search, combined multi-signal queries
- **negative queries** (no valid match should exist - expect "no sufficiently supported match")
- offline operation (airplane mode re-run of the full suite)
- incremental indexing (add one file, confirm it's searchable without a full reindex)
- app restart/recovery (kill and relaunch mid-index, confirm no lost or duplicated state)

- deletion (remove a file, confirm all associated rows are gone)

Metrics: Precision@5, Recall@10, MRR, p50/p95 latency, indexing throughput, crash/error rate - run as an automated harness in `benchmark/`, not eyeballed manually.

5.15 Exact implementation order

1. Android project foundation - *testable: app launches, empty screen renders*
2. Real file ingestion (SAF + MediaStore) - *testable: selecting a folder populates `files` rows*
3. Room database - *testable: data survives app restart*
4. Basic metadata/keyword search - *testable: "find my resume" matches by filename*
5. Image embeddings - *testable: embedding vector persisted per image*
6. Semantic image search - *testable: "temple" returns temple photos without the word "temple" in any filename*
7. OCR/document indexing - *testable: screenshot text appears in `visual_features.ocr_text`*
8. Semantic document search - *testable: "software engineering resume" matches without exact filename overlap*
9. Person registration/recognition - *testable: "Mom" query returns registered-Mom photos above the confidence threshold, and only those*
10. Hybrid retrieval (fusion + ranking) - *testable: combined query outranks single-signal matches appropriately*
11. Cross-media retrieval (Memory Object linking) - *testable: "restaurant John sent" resolves through the full evidence chain in 5.7*
12. Evidence/explanation + confidence layer - *testable: every result shows real matched signals and a confidence bucket; negative queries return "no match"*
13. Offline verification - *testable: full query suite re-run in airplane mode with identical results*
14. Performance optimization - *testable: 5.13 "Measured" column filled in against real numbers*
15. Benchmarking - *testable: automated harness produces Precision@5/Recall@10/MRR/latency report*
16. Production hardening (error states, restart recovery, deletion lifecycle) - *testable: Definition of Done checklist (Section 4) fully checked*

What must be complete before proceeding to the next stage: each numbered step's "testable" condition must pass before starting the next step - this is the same vertical-slice discipline as the blueprint's build sequence, made concrete enough to start coding directly from this spec without further architectural decisions during the event itself.

---

6. Execution Plan Mapped to the Actual Chennai Schedule

The Chennai City Battle runs a 25-hour hacking window (Sat 11:00 -> Sun 12:00), split **55% Red Light (phone-only, via Office Kit) / 45% Green Light (phone + laptop)**, with two grey evaluation blocks outside the split. Team size: solo or up to 3, each builder gets one iQOO loaner phone with Office Kit and OriginOS 6 pre-installed.

Slot	Phase	Time	Goal	Milestone

1	Green	Sat 11:00-13:00	Skeleton	Android shell, home screen, search box, local storage - build on laptop, mirror to phone via Office Kit
2	**Red**	Sat 13:00-15:30	First search (phone-only)	Ingestion + metadata + basic semantic retrieval, driven entirely from the phone through Office Kit - “find my resume” works
3	Green	Sat 15:30-16:30	Visual understanding	Image embeddings + person registration - “Mom at the temple” works
4	**Red**	Sat 16:30-19:00	Cross-media (phone-only)	OCR + screenshot/document indexing - “restaurant John sent me” works
-	Grey	Sat 19:00-22:00	Evaluation round 1	Have a working vertical slice ready to show - this is scored, not build time
5	**Red**	Sat 22:00-00:00	Performance pass	Latency, caching, memory tuning on-device
6	**Red**	Sun 00:00-01:00	Offline verification	Airplane-mode search confirmed working
7	Green (overnight)	Sun 01:00-06:30	Demo hardening	Freeze features; stress-test every demo query, restarts, empty states, crash recovery - heaviest laptop work happens here
8	**Red**	Sun 06:30-09:00	Final polish (phone-only)	Match explanations, privacy indicator, branding, final phone-side pass
-	Grey	Sun 09:00-12:00	Evaluation round 2 + final pitch	Rehearsed 3-5 minute demo, no more code

---

Priority stack under time pressure: (1) reliable live demo → (2) phone-first performance and genuine phone/camera/voice/Office-Kit usage (this is literally 25% of the score, logged by HackTracker) → (3) AI doing meaningful work → (4) UX polish → (5) architectural sophistication → (6) extra features. Never trade a higher priority for a lower one.

Logistics to lock before Sept 12: confirm team roster (up to 3), pre-install and test Office Kit (pc.vivoglobal.com) on the laptop(s) you’ll bring, and rehearse the screen-mirror + file-transfer + remote-control workflow beforehand so Red Light phases aren’t spent debugging the bridge itself.

---

7. Demo Dataset & Script

A curated 150-300 file dataset is prepared in advance and indexed using the same production pipeline used for user-selected files - curated data is not fake data; it goes through real ingestion, real embeddings, real OCR.

- **Memory A - Family:** Mom, Dad, temple, beach, birthday photos
- **Memory B - College:** project PDF, resume, presentation, notes, screenshots
- **Memory C - Chennai trip:** photos, restaurant screenshot, hotel confirmation, map screenshot, travel PDF
- **Noise:** duplicates, random screenshots, unrelated PDFs, memes - a messy dataset proves the problem is real.

The demo: three scenes, not a feature tour

Scene 1 - Personal memory. *"Show me photos of Mom at the temple."* Demonstrates person + vision + semantic retrieval.

Scene 2 - Cross-media (the hero moment). *"Find the restaurant John sent me."* Demonstrates person + OCR + screenshot + semantic retrieval - a screenshot resolving to a real file via extracted text.

Scene 3 - Privacy. Turn off the network, then run *"Find my software engineering resume."* Demonstrates document OCR + semantic retrieval + offline operation.

That's the whole demo. Don't add a fourth feature to the live run - full "everything from my Chennai trip" event understanding is a strong future capability (see roadmap) but stays out of the guaranteed demo path; if a Chennai-trip query is shown, it's the V1-realistic version (see query ladder) built from real evidence links, not a claimed event graph.

Rehearsed query ladder (for Q&A / stretch demo, beyond the three core scenes)

1. "Find my resume." 2. "Find my software engineering resume." 3. "Show me photos of Mom at the temple." 4. "Find the screenshot where John sent me the restaurant." 5. "Show me the restaurant screenshot and documents related to my Chennai trip." (a V1-realistic cross-media query - real evidence linking by shared location/date/entity, not a claimed event graph.)

---

8. Anticipated Judge Questions

- **"Isn't this just Google Photos?"** → Google Photos primarily organizes and searches a photo library. MEMORY treats photos, screenshots, and documents as connected personal evidence and retrieves them the way you actually remember them - built around on-device processing.
- **"Why on-device?"** → This is the most private data on the phone; it's also faster and works with no network.
- **"Why use an LLM at all?"** → Only to interpret ambiguous natural-language intent - retrieval itself runs on deterministic, semantic, and metadata signals.
- **"Can it scale?"** → Incremental indexing, cheap-signal-first processing, caching, and quantized/compact models.

- **“What if the AI gets it wrong?”** → It retrieves and explains its match from real evidence; it never silently edits or deletes user data.

---

9. Three Things Judges Should Remember

1. “I can talk to my phone like I talk to my memory.”
2. “It searched across my actual personal files, not just photos.”
3. “It did it privately, on the phone.”

North star: the reaction we’re designing for is *“Why doesn’t my phone already do this?”*